# CityFix

Django app for reporting and managing city issues. Users sign in with Google and have one of three roles: Citizen, City Worker or Admin.

## Getting started

### 1. Clone and create a virtual environment

```powershell
git clone <repository-url>
cd cityFix
python -m venv venv
venv\Scripts\activate
```

On macOS/Linux, activate with `source venv/bin/activate`.

If PowerShell blocks the activation script, run this once:

```powershell
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

### 2. Install dependencies

```powershell
pip install -r requirements.txt
```

### 3. Configure Google login

Create a file named `.env` in the project root (next to `manage.py`):

```
GOOGLE_CLIENT_ID=your-client-id.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=your-client-secret
```

Get the credentials from the team, or create your own in Google Cloud Console (OAuth client ID, type "Web application") with these authorized redirect URIs:

- `http://127.0.0.1:8000/accounts/google/login/callback/`
- `http://localhost:8000/accounts/google/login/callback/`

Never commit `.env`.

### 4. Create the database and an admin

```powershell
```

```
python manage.py migrate
python manage.py createsuperuser
```

The superuser logs in with email and password at `/admin/`.

### 5. Run the app and the tests

```powershell
python manage.py runserver
python manage.py test
```

Open `http://127.0.0.1:8000/`. Use the same host (`127.0.0.1` or `localhost`) every time, because sessions are not shared between them.

Google login: `http://127.0.0.1:8000/accounts/google/login/`

## Conventions

- **User model:** always reference it with `settings.AUTH_USER_MODEL`, never import `User` directly in model fields.
- **Foreign keys to users** (reporter, author, uploaded_by, worker...) must use `null=True, on_delete=models.SET_NULL`. When an account is deleted, reports and other data stay but lose their link to the user. The test `AccountDeletionContractTests` fails if a relation breaks this rule.
- **Role-based access:** use the helpers in `users/permissions.py`:

```python
from users.models import Role
from users.permissions import AdminRequiredMixin, RoleRequiredMixin, role_req

class MyView(AdminRequiredMixin, View): ...

class OtherView(RoleRequiredMixin, View):
    allowed_roles = (Role.WORKER, Role.ADMIN)

@role_required(Role.CITIZEN)
def my_function_view(request): ...
```

- **Role proxies:** `Citizen`, `CityWorker` and `Admin` in `users/models.py` share the `users_user` table. Add role-specific methods there.
- Save files as UTF-8.

## Do not commit

`venv/`, `.env`, `db.sqlite3`, `__pycache__/`. Make sure they are listed in `.gitignore`.